

Παράλληλος Προγραμματισμός 2026

2η Προγραμματιστική Εργασία

Νικόλαος Πουλόπουλος

AM: 3230170

E-mail: p3230170@aueb.gr

Εισαγωγή.....	2
Όλες οι μετρήσεις έγιναν στο παρακάτω σύστημα:.....	2
Το πρόβλημα.....	2
Υλοποιήσεις.....	3
Σειριακή υλοποίηση.....	3
Μέθοδος παραλληλοποίησης (Α: Παραλληλοποίηση με βρόχο for).....	4
Parallel for χωρίς reduction.....	4
Parallel for με reduction.....	6
Σύγκριση Parallel for χωρίς/με reduction.....	8
Μέθοδος παραλληλοποίησης (Β: Εναλλακτική δρομολόγηση επαναλήψεων).....	9
Parallel for με schedule(static).....	9
Parallel for με schedule(dynamic).....	11
Parallel for με schedule(guided).....	13
Συνολική σύγκριση static, dynamic, guided.....	14
Parallel for με schedule(static) - different f(x).....	16
Parallel for με schedule(dynamic) - different f(x).....	18
Parallel for με schedule(guided) - different f(x).....	20
Συνολική σύγκριση static, dynamic, guided.....	22
Μέθοδος παραλληλοποίησης (Γ: Παραλληλοποίηση με OpenMP tasks).....	24
Ανάλυση απόδοσης υλοποίησης με OpenMP Tasks χωρίς Αναδρομή.....	24
Ανάλυση απόδοσης υλοποίησης με OpenMP Tasks με Αναδρομή.....	30
Συγκριτική αξιολόγηση non recursive και recursive OpenMP tasks.....	34

Εισαγωγή

Το 2ο σετ ασκήσεων αφορά τον παράλληλο προγραμματισμό με OpenMP. Ζητείται η παραλληλοποίηση του υπολογισμού αριθμητικής ολοκλήρωσης μιας συνάρτησης $f(x)$ με τον Τραπεζοειδή Τύπο εξετάζοντας διαφορετικούς τρόπους διαμοιρασμού της δουλειάς και συγκρίνοντας την απόδοσή τους με τον σειριακό κώδικα.

Όλες οι μετρήσεις έγιναν στο παρακάτω σύστημα:

Όνομα υπολογιστή	Nikolas
Επεξεργαστής	AMD Ryzen 5 5500
Πλήθος πυρήνων	6
Μεταφραστής	gcc v15.2.0

Η μεταγλώττιση έγινε με τις επιλογές `-std=c++17 -pthread` χωρίς επίπεδα βελτιστοποίησης (`-O1/-O2/-O3`) !

Το πρόβλημα

Στην άσκηση αυτή ζητείται να παραλληλοποιηθεί ο υπολογισμός της αριθμητικής ολοκλήρωσης μιας συνάρτησης $f(x)$ στο διάστημα $[a,b]$ με χρήση του Τραπεζοειδούς κανόνα. Δίνεται ένας σειριακός κώδικας σε C++ που προσεγγίζει το ολοκλήρωμα με n υποδιαιρέσεις.

$a = 0$, $b = 10$ και μεγάλο N , υπολογίζει προσεγγιστικά το ολοκλήρωμα

$$\int_a^b f(x)dx \approx \frac{h}{2} \left[f(a) + f(b) + 2 \sum_{i=1}^{n-1} f(a + ih) \right], h = (b - a)/n$$

Στόχος είναι να υλοποιηθούν διαφορετικές παράλληλες εκδόσεις με χρήση οι οποίες να εκτελούν τον ίδιο υπολογισμό με διαφορετικό αριθμό νημάτων και να αξιολογείται η απόδοση.

Συγκεκριμένα υλοποιούνται και συγκρίνονται:

- Parallel for χωρίς reduction και με reduction. .
- Χρήση διαφορετικών scheduling techniques(static, dynamic, guided) και διαφορετικών chunk sizes .
- Χρήση OpenMP tasks χωρίς αναδρομή και με αναδρομή με ακανόνιστο και δυναμικά παραγόμενο φόρτο εργασίας.

Υλοποιήσεις

Σειριακή υλοποίηση

Ο σειριακός κώδικας υλοποιεί τον Τραπεζοειδή κανόνα για $f(x)=x^2$ στο $[0,10]$ με $N=10^8$ υποδιαιρέσεις και χρησιμοποιείται ως βάση για τη σύγκριση των παράλληλων εκδόσεων. Τρέχοντας τον κώδικα 20 φορές οι μετρούμενοι χρόνοι εκτέλεσης ήταν :

Εκτέλεση	Χρόνος (sec)
1η	0.391210
2η	0.390687
3η	0.391095
4η	0.391601
:	:
20η	0.391353
Μέσος	0.391721

Οδηγίες Εκτέλεσης

- **Compile:** `g++ -std=c++17 .\integration.cpp -o .\integration`
- **Run:** `.\integration.exe`

Μέθοδος παραλληλοποίησης (A: Παραλληλοποίηση με βρόχο for)

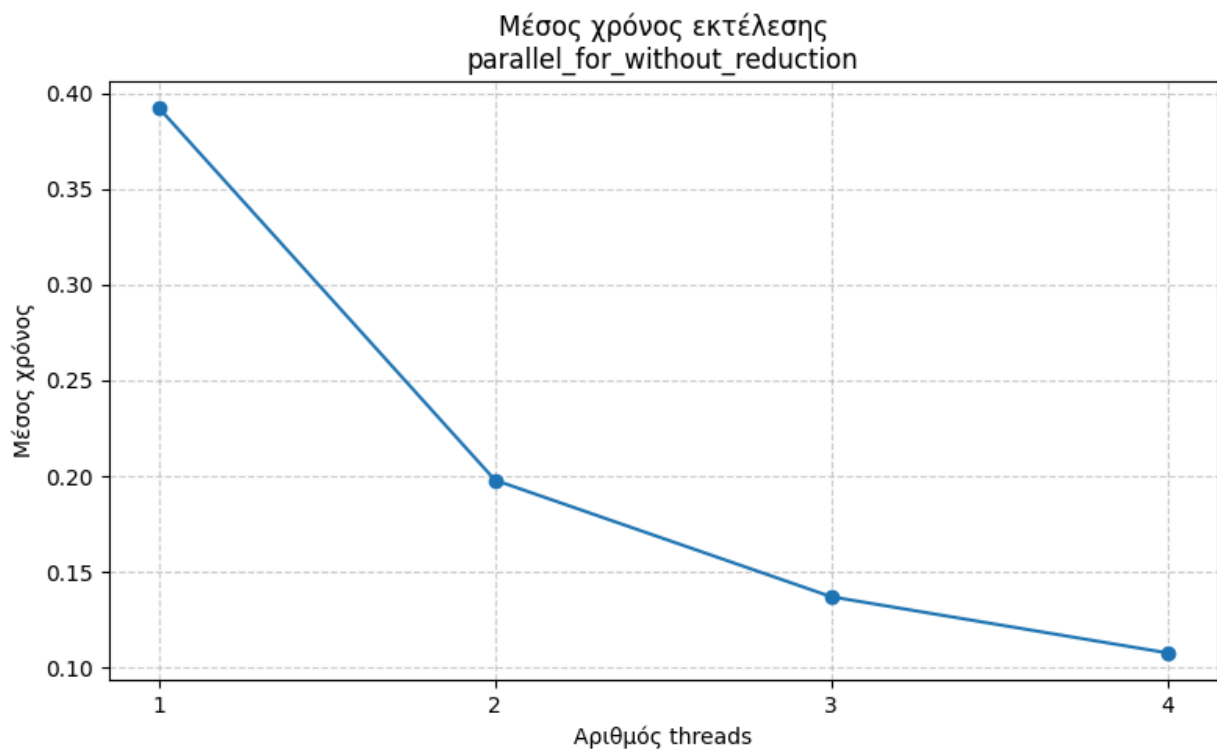
Parallel for χωρίς reduction

- Σε αυτή την υλοποίηση χρησιμοποιείται OpenMP parallel σε συνδυασμό με `#pragma omp for` για να διαμοιραστεί ο βρόχος ολοκλήρωσης στα διαθέσιμα νήματα. Κάθε νήμα υπολογίζει το μερίδιό του από τα τραπεζοειδή (ένα συνεχόμενο υποδιάστημα του $[0, N]$) και αποθηκεύει το αποτέλεσμα σε μια τοπική μεταβλητή `mysum`. Στο τέλος του βρόχου, κάθε νήμα εισέρχεται σε `#pragma omp critical` ώστε να προσθέσει το `mysum` στο κοινό `integral`.
- Ο διαμοιρασμός των επαναλήψεων γίνεται στατικά από το `omp for` και εξασφαλίζει ότι κάθε νήμα λαμβάνει περίπου τον ίδιο αριθμό επαναλήψεων

ενώ η κρίσιμη περιοχή εγγυάται ορθή πρόσβαση στο κοινό άθροισμα με κόστος συγχρονισμού κάθε φορά που ένα νήμα ενημερώνει το integral.

- Πίνακας χρόνων - Parallel for χωρίς reduction, $N = 100000000$, $f(x)=x^2$

Νήματα	1η	2η	3η	4η	5η	Μέσος όρος
1	0.3874	0.3872	0.4099	0.3889	0.3887	0.3920
2	0.1997	0.1963	0.1970	0.2002	0.1964	0.1977
3	0.1335	0.1339	0.1476	0.1358	0.1341	0.1374
4	0.1100	0.1008	0.1067	0.1135	0.1074	0.1077



- Με βάση τα νέα αποτελέσματα παρατηρούμε ότι ο μέσος χρόνος εκτέλεσης μειώνεται περίπου στο μισό όταν αυξάνουμε τα νήματα από 1 σε 2 (από

~0.392s σε ~0.198s) και συνεχίζει να βελτιώνεται καθώς φτάνουμε τα 4 νήματα (~0.108s).

- Η συμπεριφορά αυτή είναι αναμενόμενη καθώς ο omp for με στατικό διαμοιρασμό μοιράζει ισόποσα τις επαναλήψεις και κάθε επανάληψη έχει παρόμοιο φόρτο υπολογισμών $f(x)=x^2$ επομένως η παραλληλοποίηση εκμεταλλεύεται αποτελεσματικά τους διαθέσιμους πυρήνες. Παρόλα αυτά η χρήση της critical περιοχής για την ενημέρωση του integral εισάγει ένα σειριακό σημείο συγχρονισμού στο τέλος του βρόχου το οποίο περιορίζει το ιδανικό speedup ειδικά καθώς αυξάνεται ο αριθμός των νημάτων πέρα από τα 2-3 Αυτό αποτελεί και το κίνητρο για τη χρήση reduction(+:integral) στην επόμενη υλοποίηση όπου το ίδιο μοτίβο υπολογισμού μπορεί να υλοποιηθεί χωρίς ρητή κρίσιμη περιοχή βελτιώνοντας την κλιμάκωση.

Οδηγίες Εκτέλεσης

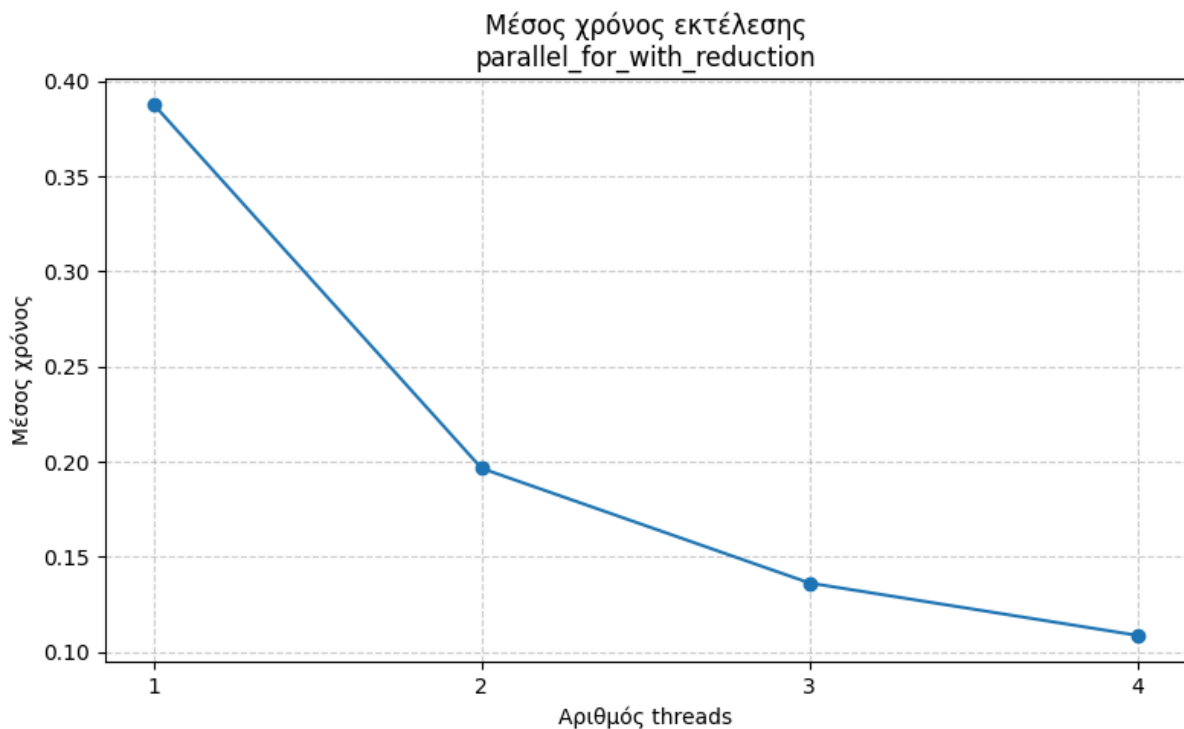
- Compile:
`g++ -std=c++17 -fopenmp parallel_for_without_reduction.cpp -o parallel_for_without_reduction`
- Run:
`./parallel_for_without_reduction num_threads`

Parallel for με reduction

- Σε αυτή την υλοποίηση χρησιμοποιείται OpenMP parallel for με reduction(+:integral) για τη συσσώρευση του αποτελέσματος. Ο βρόχος ολοκλήρωσης παραμένει ίδιος με την εκδοχή χωρίς reduction αλλά η μεταβλητή integral δηλώνεται σε reduction με αποτέλεσμα κάθε νήμα να έχει το δικό του private αντίγραφο κατά την εκτέλεση του βρόχου και όλα τα τοπικά αποτελέσματα να συνδυάζονται αυτόματα στο τέλος με άθροιση.
- Με αυτό τον τρόπο αποφεύγεται η ρητή χρήση #pragma omp critical για την ενημέρωση του κοινόχρηστου αθροίσματος και μειώνεται το synchronization overhead ιδιαίτερα όταν αυξάνεται ο αριθμός των νημάτων. Ο διαμοιρασμός των επαναλήψεων εξακολουθεί να είναι στατικός μέσω του omp for συνεπώς το load είναι ισοκαταμεμημένο αφού κάθε επανάληψη έχει παρόμοιο υπολογιστικό κόστος για $f(x)=x^2$

- Πίνακας χρόνων (Parallel for με reduction, N = 100000000), $f(x)=x^2$

Νήματα	1η	2η	3η	4η	5η	Μέσος όρος
1	0.3865	0.3902	0.3883	0.3884	0.3858	0.3878
2	0.1970	0.1972	0.1967	0.1961	0.1956	0.1967
3	0.1319	0.1325	0.1311	0.1427	0.1429	0.1366
4	0.0994	0.1055	0.1089	0.1128	0.1168	0.1083



- Από τα νέα αποτελέσματα βλέπουμε ότι ο μέσος χρόνος εκτέλεσης μειώνεται από περίπου 0.388 s με 1 νήμα σε ~0.197 s με 2 νήματα και συνεχίζει να

βελτιώνεται σε ~0.137s με 3 νήματα και ~0.108s με 4 νήματα. Η κλιμάκωση είναι σαφώς καλύτερη σε σχέση με την εκδοχή χωρίς reduction.

- Σε σύγκριση με την εκδοχή parallel for χωρίς reduction όπου η ενημέρωση του integral γίνεται μέσα από critical περιοχή η εκδοχή με reduction επιτυγχάνει παρόμοιους χρόνους για 1 νήμα αλλά εμφανώς καλύτερη κλιμάκωση από 2 έως 4 νήματα. Αυτό δείχνει στην πράξη ότι ο reduction είναι ο προτιμώμενος τρόπος για αυτό το είδος παραλληλισμού.

Οδηγίες Εκτέλεσης

- Compile:
g++ -std=c++17 -fopenmp parallel_for_with_reduction.cpp -o parallel_for_with_reduction
- Run:
./parallel_for_with_reduction num_threads

Σύγκριση Parallel for χωρίς/με reduction

Στον παρακάτω πίνακα φαίνονται οι μέσοι χρόνοι εκτέλεσης για τις δύο υλοποιήσεις parallel for (χωρίς reduction με critical και με reduction(+:integral)) για $N=10^8$ και $f(x)=x^2$:

Νήματα	Χωρίς reduction (sec)	Με reduction (sec)
1	0.3920	0.3878
2	0.1977	0.1967
3	0.1374	0.1366
4	0.1077	0.1083

Παρατηρούμε ότι για 1 νήμα οι χρόνοι των δύο εκδοχών είναι πρακτικά ίδιοι (0.3920s vs 0.3878s) όπως αναμενόταν και το overhead από critical ή reduction είναι πολύ μικρό σε σχέση με το κόστος του ίδιου του υπολογισμού του ολοκληρώματος. Για 2 έως 4 νήματα οι δύο υλοποιήσεις δίνουν επίσης πολύ κοντινούς μέσους χρόνους με την έκδοση με reduction να είναι οριακά ταχύτερη στα 2-3 νήματα και ελάχιστα πιο αργή στα 4 νήματα με διαφορές της τάξης λίγων χιλιοστών του δευτερολέπτου.

Η συμπεριφορά αυτή εξηγείται από το γεγονός ότι στην υλοποίηση χωρίς reduction κάθε νήμα υπολογίζει το δικό του mysum και μπαίνει στην κρίσιμη περιοχή μόνο μία φορά στο τέλος του βρόχου οπότε το σειριακό τμήμα λόγω critical είναι πολύ μικρό. Αντίστοιχα στην υλοποίηση με reduction η OpenMP αναλαμβάνει αυτόματα τη

δημιουργία και συγχώνευση των private αντιγράφων της integral εισάγοντας ένα μικρό αλλά σταθερό overhead σε κάθε run το οποίο όμως για αυτό το πρόβλημα παραμένει συγκρίσιμο με το κόστος της μοναδικής εισόδου σε critical ανά νήμα.

Θεωρητικά για μεγαλύτερο αριθμό νημάτων ή για σενάρια όπου η ενημέρωση του κοινόχρηστου αθροίσματος γίνεται πολύ συχνά μέσα στο loop η χρήση reduction είναι πιο κλιμακώσιμη και αναμένεται να υπερτερεί καθώς περιορίζει το σειριακό σημείο συγχρονισμού και μεταφέρει τη μείωση στο OpenMP runtime. Στο συγκεκριμένο όμως πρόβλημα με ένα critical ανά νήμα και έως 4 threads οι δύο υλοποιήσεις έχουν σχεδόν ταυτόσημη συμπεριφορά και τα πλεονεκτήματα της reduction φαίνονται περισσότερο σε επίπεδο καθαρότητας και επεκτασιμότητας του κώδικα παρά σε διαφορά χρόνου για τα παρόντα πειράματα.

Μέθοδος παραλληλοποίησης(B: Εναλλακτική δρομολόγηση επαναλήψεων)

Για την αξιολόγηση χρησιμοποιήθηκαν οι τιμές:

- chunk = 1, 10, 100, 1000
- num_threads = 1, 2, 3, 4

και κάθε συνδυασμός εκτελέστηκε 5 φορές με τον μέσο χρόνο να καταγράφεται στους πίνακες.

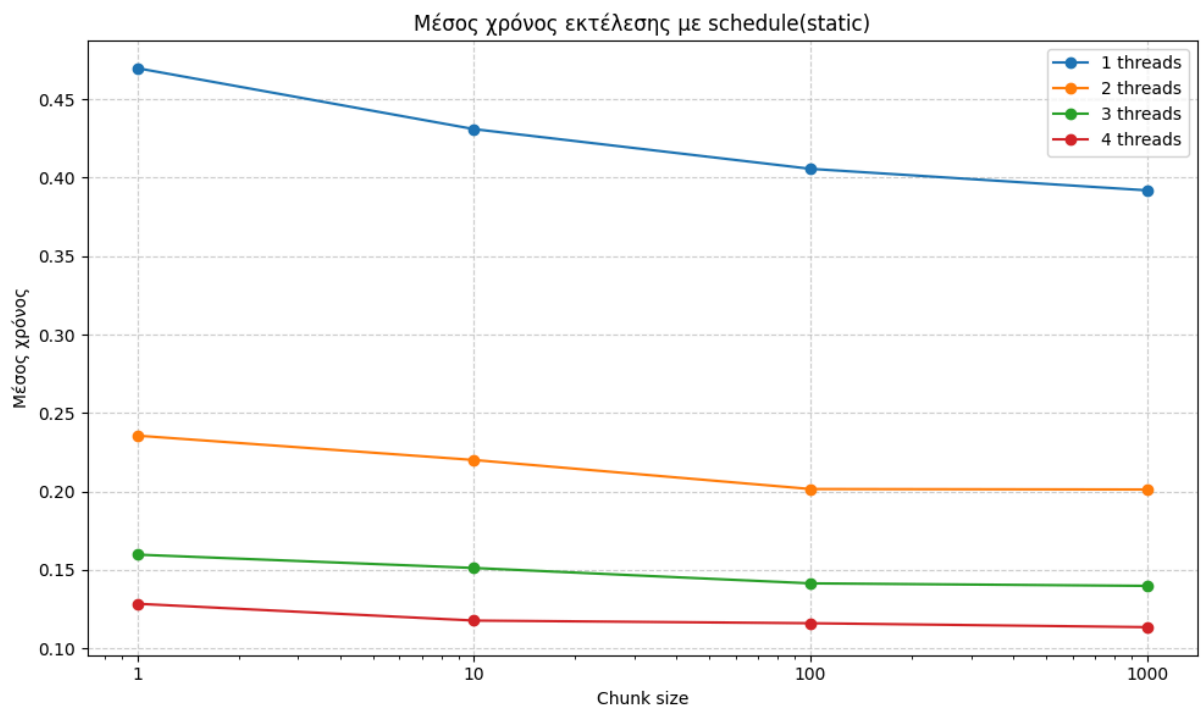
Parallel for με schedule(static)

- Στην πρώτη υλοποίηση αξιοποιείται η οδηγία OpenMP parallel for σε συνδυασμό με reduction(+:integral) ενώ ο διαμοιρασμός των επαναλήψεων γίνεται με schedule(static, chunk). Στο static scheduling οι επαναλήψεις κατανέμονται εκ των προτέρων στα νήματα, με αποτέλεσμα ελάχιστο runtime overhead. Η προσέγγιση αυτή είναι ιδιαίτερα αποδοτική όταν κάθε επανάληψη έχει παρόμοιο υπολογιστικό κόστος όπως συμβαίνει εδώ με τη συνάρτηση $f(x)=x^2$

- Για την αξιολόγηση της απόδοσης πραγματοποιήθηκαν πειράματα με τιμές chunk ίσες με 1, 10, 100 και 1000 καθώς και με αριθμό νημάτων από 1 έως 4. Για κάθε συνδυασμό εκτελέστηκαν 5 επαναλήψεις και υπολογίστηκε ο μέσος χρόνος εκτέλεσης.

- Πίνακας χρόνων schedule(static), $f(x)=x^2$

Threads	chunk=1	chunk=10	chunk=100	chunk=1000
1	0.4697	0.4309	0.4056	0.3919
2	0.2356	0.2201	0.2016	0.2013
3	0.1598	0.1513	0.1415	0.1399
4	0.1285	0.1178	0.1161	0.1136



- Από τα αποτελέσματα φαίνεται ότι ο static παρουσιάζει σταθερή και προβλέψιμη συμπεριφορά για όλα τα chunk sizes με τις διαφορές στους χρόνους εκτέλεσης να είναι σχετικά μικρές. Για 1 νήμα ο μέσος χρόνος μειώνεται από περίπου 0.47s για chunk=1 σε περίπου 0.39s για chunk=1000 δείχνοντας ότι το μεγαλύτερο chunk μειώνει ελαφρώς το overhead της

δρομολόγησης και βελτιώνει την τοπικότητα μνήμης. Για 2-4 νήματα οι χρόνοι πέφτουν σημαντικά (γύρω στα 0.20s για 2 threads και ~0.11-0.14s για 3-4 threads) επιβεβαιώνοντας ότι ο παραλληλισμός εκμεταλλεύεται αποτελεσματικά τους πυρήνες ακόμη και χωρίς δυναμικό scheduling.

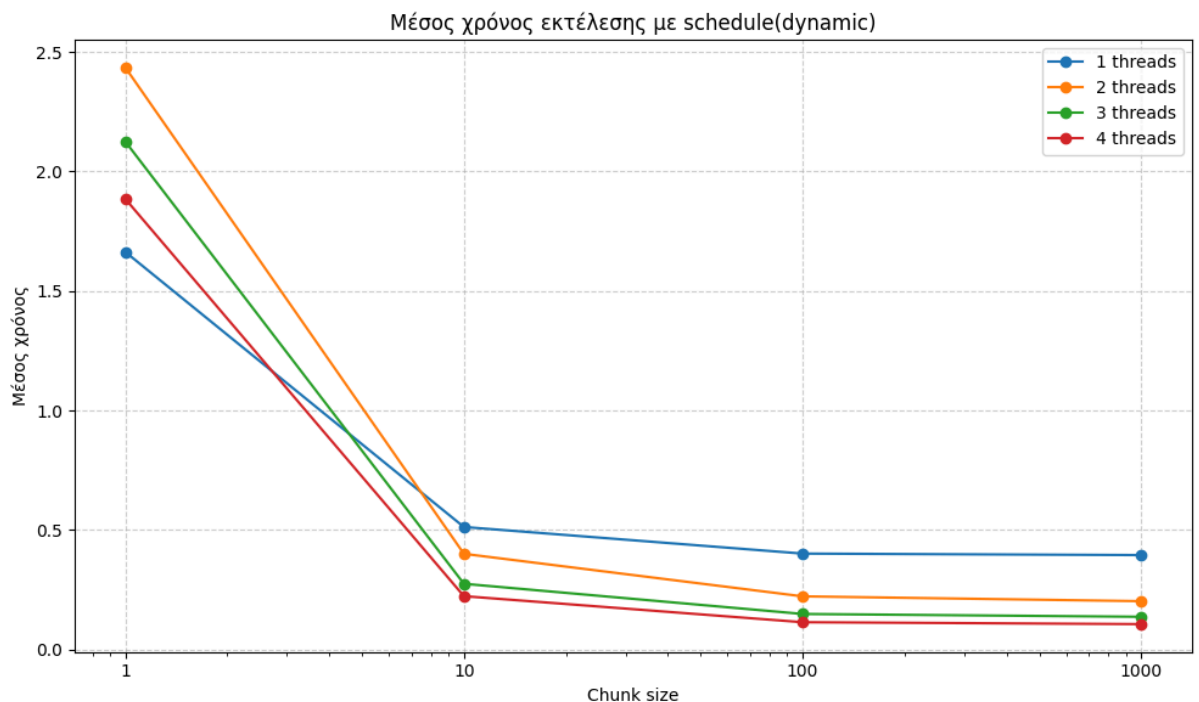
- Η καλή αυτή συμπεριφορά εξηγείται από το ότι η συνάρτηση $f(x)=x^2$ έχει ομοιόμορφο υπολογιστικό κόστος σε κάθε επανάληψη, άρα δεν δημιουργείται load imbalance ανάμεσα στα threads. Επομένως το static scheduling αποτελεί φυσική και αποδοτική επιλογή καθώς αποφεύγει το επιπλέον κόστος συγχρονισμού και ανάθεσης εργασίας που συναντάται στις δυναμικές πολιτικές scheduling χωρίς να θυσιάζει την ισοκατανομή του φόρτου για αυτό το συγκεκριμένο πρόβλημα.

Parallel for με schedule(dynamic)

- Στη δεύτερη υλοποίηση χρησιμοποιείται η ίδια δομή parallel for με reduction(+:integral) αλλά οι επαναλήψεις κατανέμονται με schedule(dynamic, chunk). Στην πολιτική αυτή τα chunks δεν ανατίθενται εκ των προτέρων κάθε νήμα μόλις ολοκληρώσει το τρέχον chunk του ζητά νέο chunk από τον runtime scheduler. Αυτό επιτρέπει καλύτερο load balancing σε περιπτώσεις άνισου φόρτου με κόστος αυξημένου scheduling overhead.
- Για να εξεταστεί η επίδραση του chunk στο κόστος του δυναμικού scheduling εκτελέστηκαν τα ίδια πειράματα όπως και για το static για 1-4 νήματα και για chunk = 1, 10, 100, 1000. Για κάθε συνδυασμό μετρήθηκαν 5 εκτελέσεις και υπολογίστηκε ο μέσος χρόνος.
- Πίνακας χρόνων schedule(dynamic), $f(x)=x^2$

Threads	chunk=1	chunk=10	chunk=100	chunk=1000
1	1.664	0.513	0.402	0.395
2	2.436	0.401	0.223	0.203
3	2.126	0.275	0.149	0.137

4	1.886	0.224	0.115	0.107
---	-------	-------	-------	-------

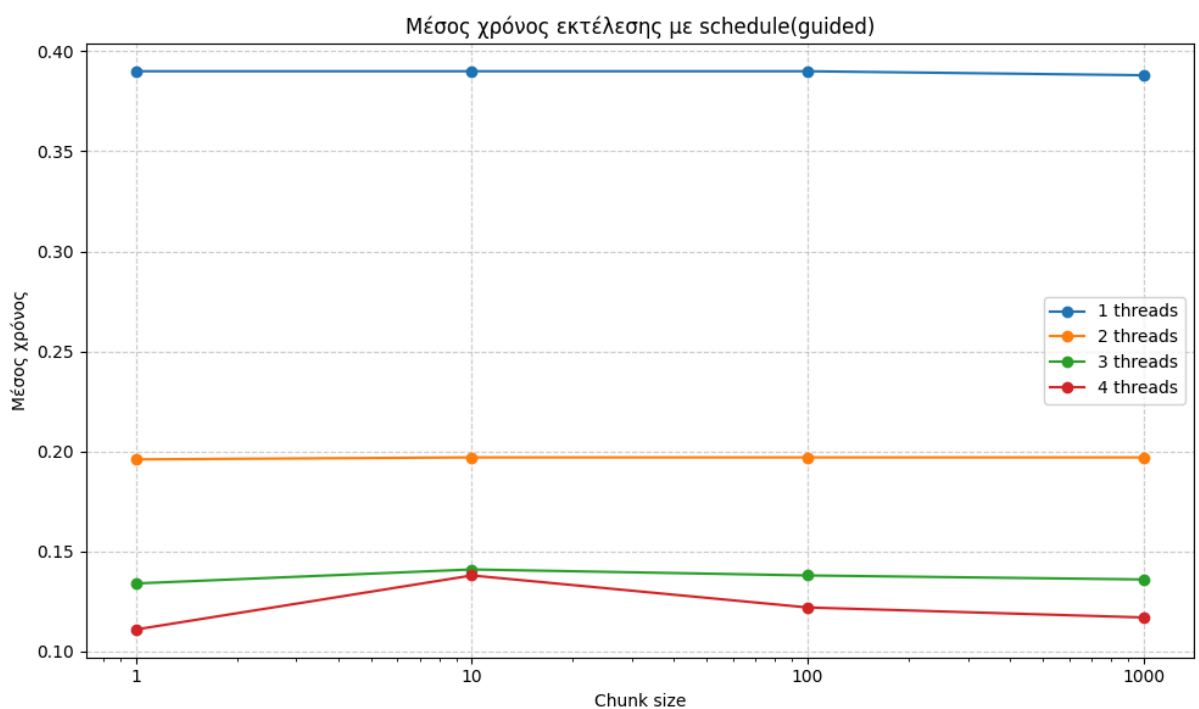


- Τα αποτελέσματα δείχνουν καθαρά ότι το schedule(dynamic) μπορεί να εισάγει σημαντικό overhead όταν το chunk size είναι πολύ μικρό. Για chunk=1, οι χρόνοι εκτέλεσης είναι περίπου 1.7s για 1 νήμα και πάνω από 1.8-2.4s για 2-4 νήματα δηλαδή πολύ μεγαλύτεροι από τους αντίστοιχους χρόνους με schedule(static). Ο λόγος είναι ότι κάθε επανάληψη του βρόχου αντιστοιχεί σε ένα chunk άρα τα νήματα επιστρέφουν συνεχώς στον scheduler για να ζητήσουν επόμενη εργασία και το runtime overhead κυριαρχεί.
- Όσο αυξάνεται το chunk size ο αριθμός των δυναμικών αναθέσεων μειώνεται και οι χρόνοι πέφτουν σημαντικά. Για chunk=10 οι χρόνοι κατεβαίνουν στο 0.22-0.51s, ενώ για chunk=100 και chunk=1000 οι επιδόσεις πλησιάζουν πολύ αυτές του static scheduling για 4 νήματα ο μέσος χρόνος είναι περίπου 0.115s με chunk=100 και 0.107s με chunk=1000 πολύ κοντά στα αντίστοιχα νούμερα του static. Ωστόσο δεν παρατηρείται ουσιαστικό πλεονέκτημα υπέρ του dynamic καθώς το workload είναι ήδη απολύτως ομοιόμορφο ($f(x)=x^2$).
- Συνεπώς για τη συγκεκριμένη συνάρτηση $f(x)=x^2$ και τον συγκεκριμένο βρόχο ολοκλήρωσης το dynamic scheduling επιβαρύνει την εκτέλεση όταν χρησιμοποιούνται πολύ μικρά chunks ενώ για μεγαλύτερα chunk sizes καταλήγει να έχει παρόμοια συμπεριφορά με το static χωρίς όμως να προσφέρει πρόσθετο όφελος.

Parallel for με schedule(guided)

- Στην τρίτη υλοποίηση χρησιμοποιείται `schedule(guided, chunk)`. Η `guided` μοιάζει με τη `dynamic` αλλά τα αρχικά `chunks` είναι μεγάλα και στη συνέχεια μειώνονται προοδευτικά καθώς προχωρά η εκτέλεση μέχρι να φτάσουν στο ελάχιστο μέγεθος που ορίζεται από το `chunk`. Έτσι επιδιώκει να ισορροπήσει ανάμεσα στο χαμηλό `overhead` των μεγάλων `chunks` στην αρχή και στο καλύτερο `load balancing` των μικρότερων `chunks` προς το τέλος του βρόχου.
- Οι μετρήσεις έγιναν για 1-4 νήματα και για `chunk = 1, 10, 100, 1000` με 5 επαναλήψεις ανά συνδυασμό και καταγραφή του μέσου χρόνου.
- Πίνακας χρόνων `schedule(guided)`, $f(x)=x^2$

Threads	chunk=1	chunk=10	chunk=100	chunk=1000
1	0.390	0.390	0.390	0.388
2	0.196	0.197	0.197	0.197
3	0.134	0.141	0.138	0.136
4	0.111	0.138	0.122	0.117

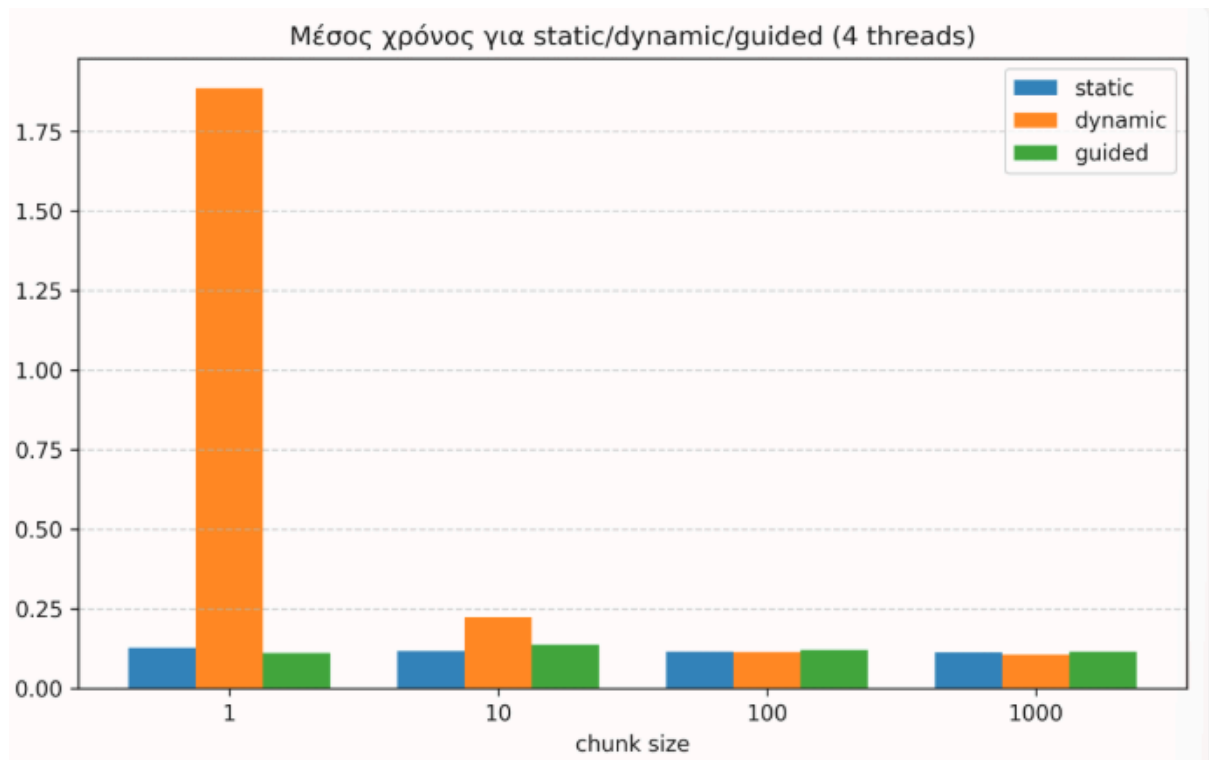
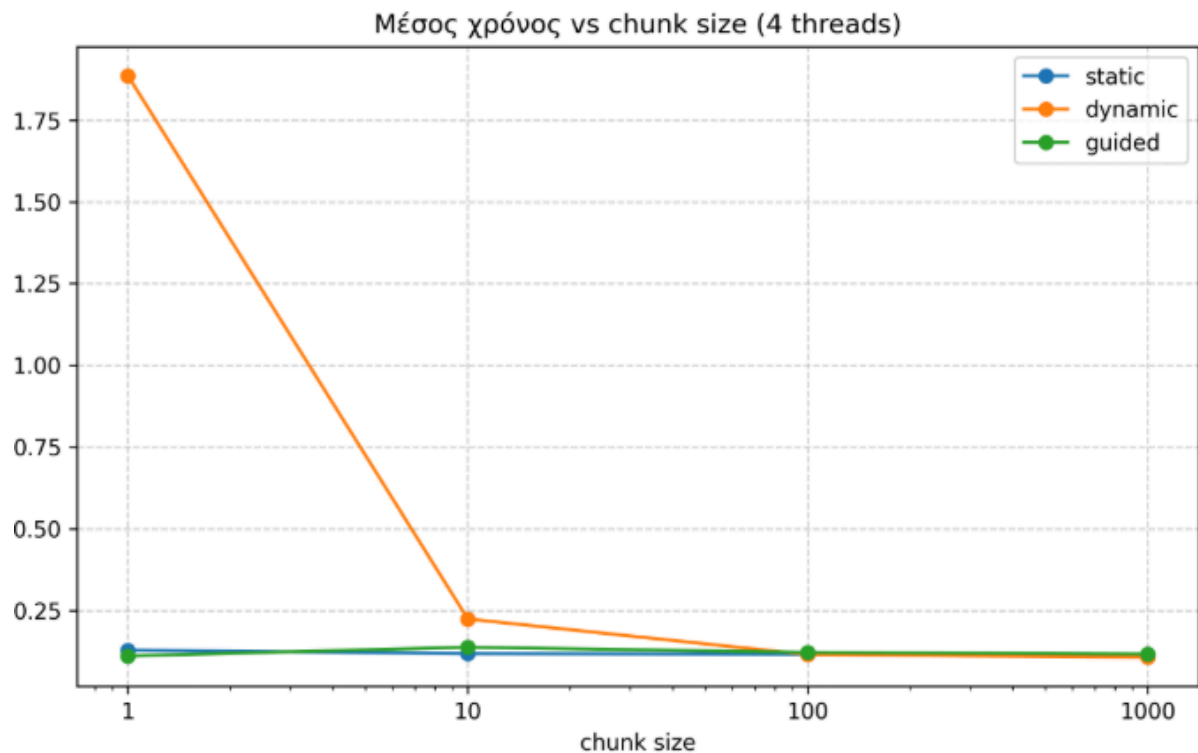


- Η συμπεριφορά αυτή είναι σύμφωνη με τη θεωρία το guided ξεκινά με μεγάλα chunks και άρα χρειάζεται λιγότερες δυναμικές αναθέσεις συνολικά μειώνοντας το scheduling overhead σε σχέση με dynamic με πολύ μικρό chunk. Στο συγκεκριμένο πρόβλημα, όπου όλες οι επαναλήψεις έχουν σχεδόν ίδιο υπολογιστικό κόστος ($f(x)=x^2$) δεν υπάρχει ουσιαστικό load imbalance και επομένως το guided δεν προσφέρει μετρήσιμο πλεονέκτημα έναντι του static αλλά λειτουργεί ως μια ασφαλής δυναμική επιλογή αποφεύγει τα ακραία του dynamic με πολύ μικρά chunks ενώ παραμένει κοντά στις επιδόσεις του static scheduling.

Συνολική σύγκριση static, dynamic, guided

Για να γίνει πιο ξεκάθαρη η σύγκριση, στον παρακάτω πίνακα παρουσιάζονται οι μέσοι χρόνοι εκτέλεσης για 4 threads και για διαφορετικά chunk sizes (1, 10, 100, 1000) για τους τρεις εναλλακτικές δρομολόγησης (static, dynamic, guided).

Schedule	chunk=1	chunk=10	chunk=100	chunk=1000
static	0.129	0.118	0.116	0.114
dynamic	1.886	0.224	0.115	0.107
guided	0.111	0.138	0.122	0.117



- Η συνολική εικόνα επιβεβαιώνει ότι για ομοιόμορφο workload όπως η ολοκλήρωση της $f(x)=x^2$ η πολιτική `schedule(static)` είναι εξαιρετικά αποδοτική για 4 νήματα δίνει χρόνους γύρω στα 0.11-0.13s για όλα τα chunk sizes και δεν επηρεάζεται ιδιαίτερα από την επιλογή του chunk χάρη στο πολύ χαμηλό runtime overhead και την ισοκατανεμημένη ανάθεση επαναλήψεων στα threads. Η πολιτική `schedule(guided)` παρουσιάζει επίσης πολύ καλή

συμπεριφορά και βρίσκεται πολύ κοντά στο static ειδικά για $\text{chunk}=1$ και $\text{chunk}=1000$ όπου οι χρόνοι είναι της τάξης των 0.11-0.12s, δείχνοντας ότι ο μηχανισμός των αρχικά μεγάλων chunks περιορίζει σημαντικά το scheduling overhead.

- Αντίθετα το `schedule(dynamic)` αποδεικνύεται ιδιαίτερα ευαίσθητο στην επιλογή του chunk size. Για $\text{chunk}=1$ ο μέσος χρόνος εκτέλεσης εκτοξεύεται περίπου στα 1.9s δηλαδή περισσότερο από τάξη μεγέθους χειρότερος από `static/guided` επειδή κάθε επανάληψη αντιστοιχεί σε ένα μικροσκοπικό chunk και ο scheduler καλείται πολύ συχνά. Όταν το chunk αυξηθεί σε 10, 100 ή 1000 το scheduling overhead μειώνεται οι χρόνοι κατεβαίνουν στο 0.22-0.11s και το dynamic πλησιάζει τις επιδόσεις των άλλων δύο τρόπων χωρίς όμως να τις ξεπερνά.
- Συνεπώς για τη συνάρτηση $f(x)=x^2$ και τον συγκεκριμένο βρόχο ολοκλήρωσης το βασικό συμπέρασμα είναι ότι η static είναι η πιο φυσική και αποδοτική επιλογή η guided αποτελεί μια πολύ καλή εναλλακτική με συμπεριφορά σχεδόν ισοδύναμη με το static ενώ η dynamic πρέπει να χρησιμοποιείται με προσοχή και αποφυγή πολύ μικρών chunks ειδικά όταν το workload είναι ομοιόμορφο και δεν δικαιολογεί το επιπλέον scheduling overhead.

Οδηγίες Εκτέλεσης

- **Compile:** `g++ -std=c++17 -fopenmp integration_openmp_schedules_fx.cpp -o integration_openmp_schedules_fx`
- **Run:** `./integration_openmp_schedules_fx <static|dynamic|guided> <chunk> <num_threads>`

Parallel for με `schedule(static)` - different $f(x)$

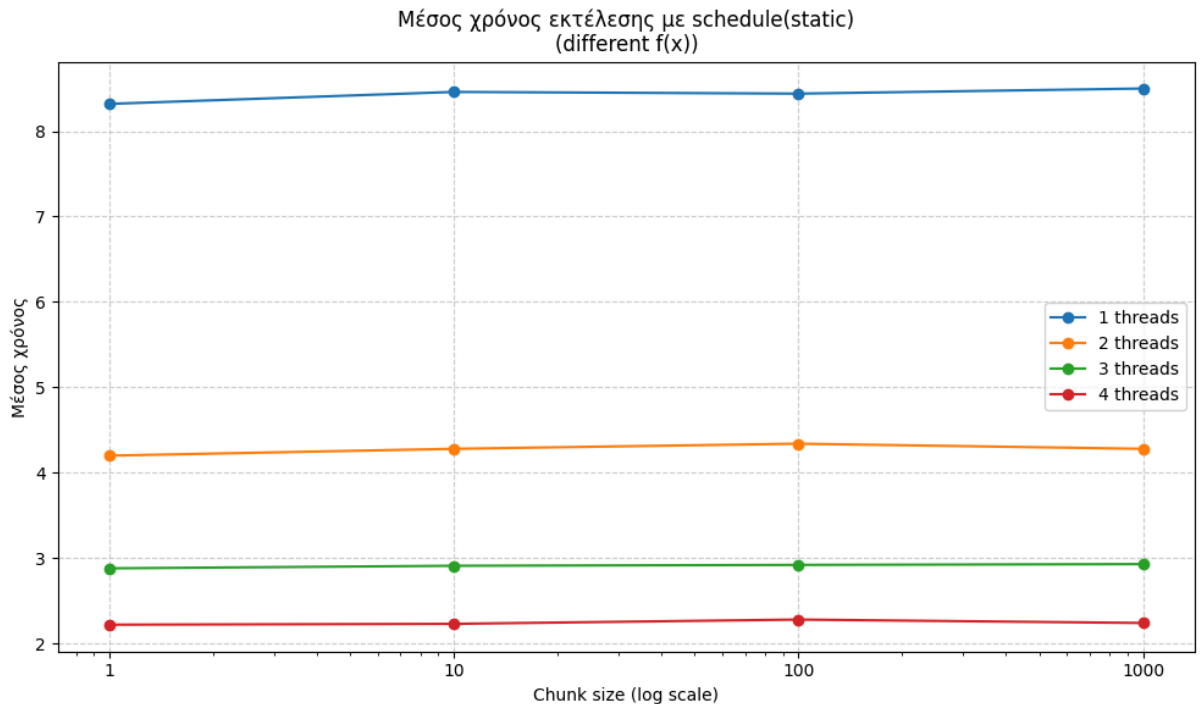
- Στην πρώτη υλοποίηση αξιοποιείται η οδηγία OpenMP `parallel for` με `reduction(+:integral)` και πολιτική `schedule(static, chunk)`, όπως και στο ομοιόμορφο σενάριο. Στο static scheduling οι επαναλήψεις κατανέμονται στα νήματα σε συνεχόμενα blocks με αποτέλεσμα πολύ μικρό runtime overhead.

Ωστόσο όταν το κόστος των επαναλήψεων διαφέρει σημαντικά, όπως στη συνάρτηση

$$f(x) = \begin{cases} \sin^2(x) \sqrt{x+1}, & \text{αν } \lfloor x \rfloor \text{ είναι άρτιος} \\ x^2, & \text{αν } \lfloor x \rfloor \text{ είναι περιττός} \end{cases}$$

- η στατική ανάθεση μπορεί να οδηγήσει σε ανισοκατανομή φόρτου καθώς κάποια threads πέφτουν σε περιοχές του διαστήματος με περισσότερες ακριβές κλήσεις της $f(x)$ ενώ άλλα ολοκληρώνουν νωρίτερα και μένουν ανενεργά.
- Για την αξιολόγηση της απόδοσης με τη different $f(x)$ χρησιμοποιήθηκαν ξανά $\text{chunk} = 1, 10, 100, 1000$ και $\text{threads} = 1-4$ με 5 επαναλήψεις ανά περίπτωση
- Πίνακας χρόνων $\text{schedule}(\text{static})$,

Threads	chunk=1	chunk=10	chunk=100	chunk=1000
1	8.32	8.46	8.44	8.50
2	4.20	4.28	4.34	4.28
3	2.88	2.91	2.92	2.93
4	2.22	2.23	2.28	2.24



- Από τα αποτελέσματα φαίνεται ότι οι χρόνοι εκτέλεσης με schedule(static) είναι σημαντικά μεγαλύτεροι από αυτούς του ομοιόμορφου σεναρίου με $f(x)=x^2$ (της τάξης των 8-8.5s για 1 νήμα αντί για ~0.39s) λόγω του αυξημένου υπολογιστικού κόστους της συνάρτησης και της ανισοκατανομής των βαριών επαναλήψεων ανάμεσα στα threads. Παρότι υπάρχει σαφής speedup καθώς αυξάνεται ο αριθμός των νημάτων (από ~8.3-8.5s με 1 thread σε ~2.2-2.3s με 4 threads) η κλιμάκωση δεν είναι ιδανική.
- Οι διαφορές μεταξύ των διαφορετικών chunk sizes παραμένουν σχετικά μικρές κάτι που επιβεβαιώνει ότι το overhead του static scheduling είναι χαμηλό και πρακτικά ανεξάρτητο από το chunk. Ωστόσο η στατική πολιτική δεν μπορεί να διορθώσει το load imbalance που προκαλεί η ανώμαλη $f(x)$ ανεξάρτητα από το chunk size, τα blocks που κατανέμονται.

Parallel for με schedule(dynamic) - different $f(x)$

- Στη δεύτερη υλοποίηση χρησιμοποιείται schedule(dynamic, chunk). Οι επαναλήψεις δεν ανατίθενται και κάθε νήμα μόλις ολοκληρώσει το τρέχον chunk ζητά νέο chunk από τον scheduler επιτρέποντας στα πιο γρήγορα threads να αναλάβουν επιπλέον εργασία και να μειώσουν το load imbalance. Το τίμημα είναι αυξημένο runtime overhead ειδικά όταν το chunk είναι πολύ μικρό καθώς ο scheduler καλείται πολύ συχνά.

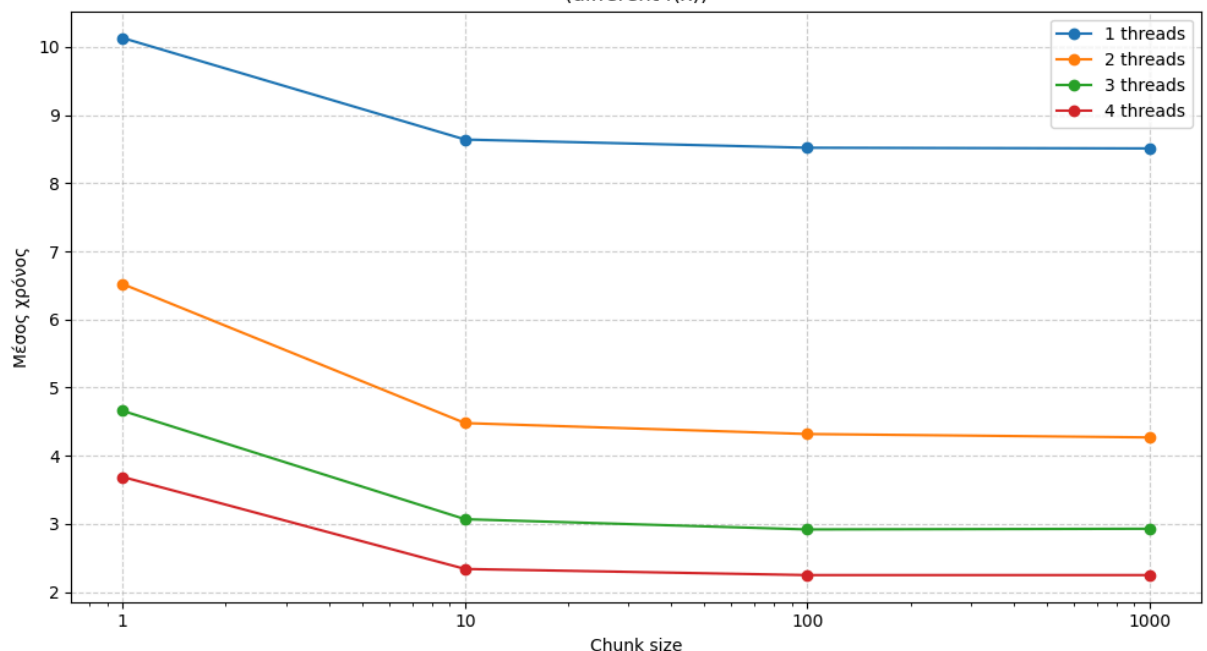
- Για την αξιολόγηση της απόδοσης με τη different f(x) χρησιμοποιήθηκαν οι ίδιες παράμετροι όπως στο static chunk = 1, 10, 100, 1000 και threads = 1-4 με 5 επαναλήψεις ανά συνδυασμό και καταγραφή του μέσου χρόνου.

- Πίνακας χρόνων schedule(dynamic)

$$f(x) = \begin{cases} \sin^2(x) \sqrt{x+1}, & \text{αν } \lfloor x \rfloor \text{ είναι άρτιος} \\ x^2, & \text{αν } \lfloor x \rfloor \text{ είναι περιττός} \end{cases}$$

Threads	chunk=1	chunk=10	chunk=100	chunk=1000
1	10.13	8.64	8.52	8.51
2	6.52	4.48	4.32	4.27
3	4.66	3.07	2.92	2.93
4	3.69	2.34	2.25	2.25

Μέσος χρόνος εκτέλεσης με schedule(dynamic)
(different f(x))



- Για chunk = 1 παρατηρείται πολύ μεγάλος χρόνος εκτέλεσης περίπου 10.1s με 1 νήμα και 6.5-4.7-3.7s για 2-4 νήματα που είναι σαφώς χειρότεροι από τους αντίστοιχους χρόνους με schedule(static). Αυτό οφείλεται στο υπερβολικό

scheduling overhead καθώς κάθε επανάληψη αντιστοιχεί σε ένα chunk και τα νήματα ζητούν συνεχώς νέα εργασία από τον runtime scheduler. Όσο αυξάνεται το chunk ο αριθμός των δυναμικών αναθέσεων μειώνεται και οι χρόνοι πέφτουν σημαντικά για chunk = 100 και 4 threads ο μέσος χρόνος είναι περίπου 2.25s, πολύ κοντά στον αντίστοιχο χρόνο του static (~2.28 s) ενώ για 2-3 threads το dynamic γίνεται ελαφρώς καλύτερο από το static ειδικά για chunk = 10-100 καθώς μοιράζει πιο ομοιόμορφα τις βαριές επαναλήψεις.

- Συνολικά η πολιτική dynamic αποδίδει καλύτερα από τη static στο ανισοκαταμεμημένο workload όταν επιλεγεί κατάλληλο chunk (περίπου 10-100 σε αυτό το πείραμα) επειδή μειώνει ουσιαστικά το load imbalance και βελτιώνει τη χρήση των νημάτων.

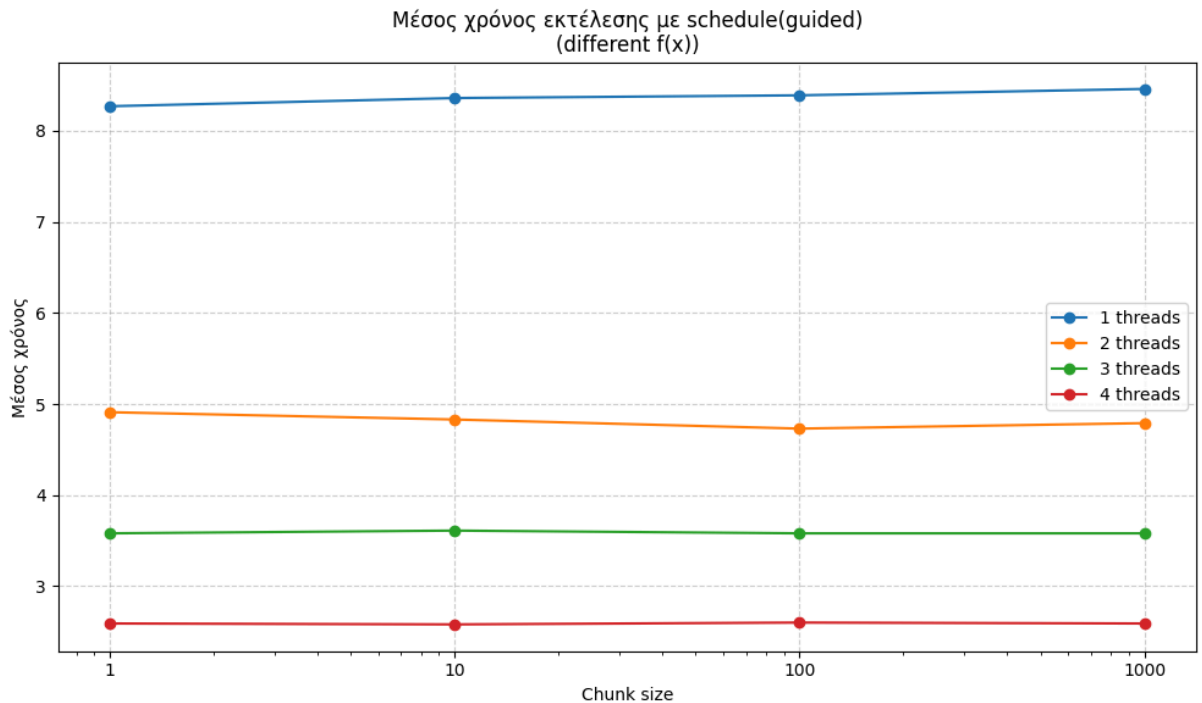
Parallel for με schedule(guided) - different f(x)

- Στην τρίτη υλοποίηση χρησιμοποιείται schedule(guided, chunk). Η πολιτική guided μοιάζει με τη dynamic αλλά τα αρχικά chunks είναι μεγάλα και μειώνονται σταδιακά όσο απομένουν λιγότερες επαναλήψεις μέχρι να φτάσουν στο ελάχιστο μέγεθος που ορίζει το chunk. Στόχος είναι να μειωθεί ο αριθμός δυναμικών αναθέσεων (άρα και το overhead) στην αρχή αλλά να διατηρηθεί ικανοποιητικό load balancing προς το τέλος της εκτέλεσης κάτι που είναι ιδιαίτερα χρήσιμο σε workloads με άνισο κόστος ανά επανάληψη.
- Για τη different f(x) εκτελέστηκαν πειράματα για threads = 1-4 και για chunk = 1, 10, 100, 1000 με 5 επαναλήψεις ανά συνδυασμό και καταγραφή του μέσου χρόνου.
- Πίνακας χρόνων schedule(**guided**)

$$f(x) = \begin{cases} \sin^2(x) \sqrt{x+1}, & \text{αν } \lfloor x \rfloor \text{ είναι άρτιος} \\ x^2, & \text{αν } \lfloor x \rfloor \text{ είναι περιττός} \end{cases}$$

Threads	chunk=1	chunk=10	chunk=100	chunk=1000
1	8.27	8.36	8.39	8.46
2	4.91	4.83	4.73	4.79

3	3.58	3.61	3.58	3.58
4	2.59	2.58	2.60	2.59

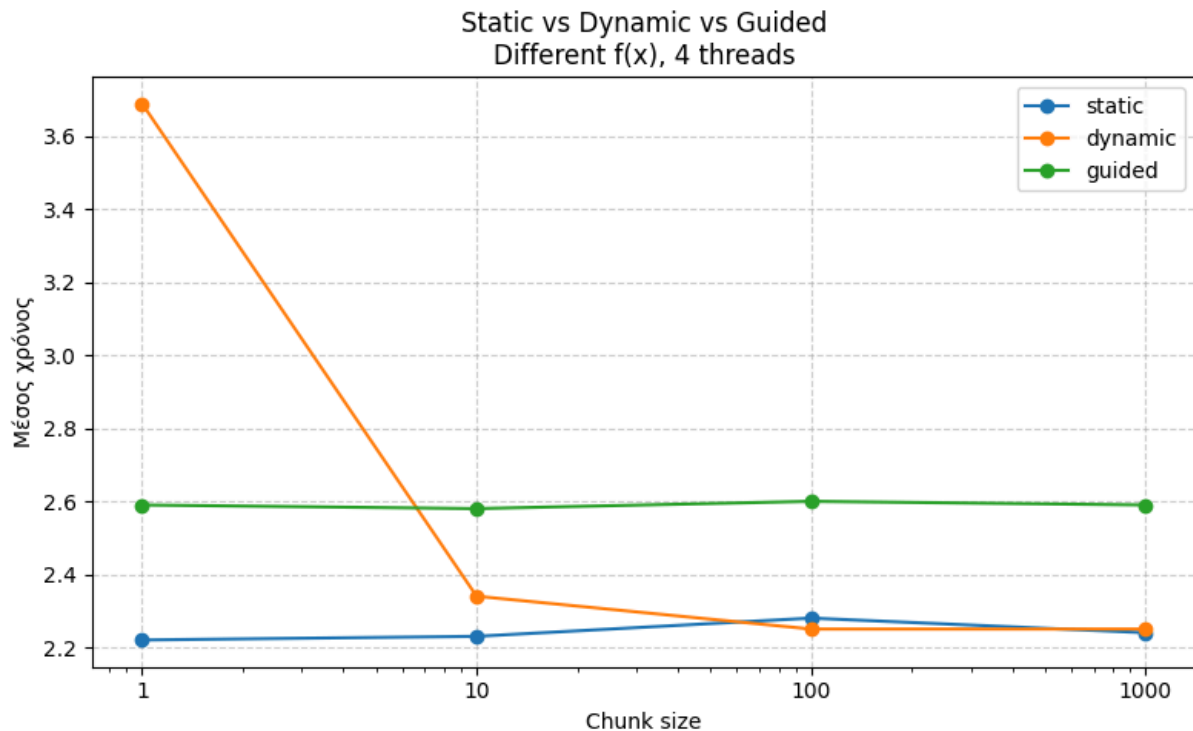


- Τα αποτελέσματα δείχνουν ότι το guided έχει επιδόσεις σαφώς καλύτερες από το static και από το dynamic με κακό chunk και αρκετά σταθερές ως προς το chunk size. Για 1 νήμα οι χρόνοι βρίσκονται γύρω στα 8.3-8.5s δηλαδή ελαφρώς καλύτεροι από το static (~8.3-8.5s) και αισθητά καλύτεροι από το dynamic με chunk=1 (~10.1s) επειδή τα μεγάλα αρχικά chunks περιορίζουν το scheduling overhead. Για 3-4 νήματα οι χρόνοι κυμαίνονται περίπου στα 3.6s και 2.6s αντίστοιχα, με πολύ μικρή ευαισθησία στο chunk σε επίπεδα που είναι καλύτερα από static (2.9-2.3s) αλλά και συγκρίσιμα dynamic (π.χ. 2.25 s με 4 threads και chunk=100).
- Ιδιαίτερα θετικό είναι ότι το guided δεν εμφανίζει την ακραία ευαισθησία στο chunk size που βλέπουμε στο dynamic όπου οι διαφορές μεταξύ chunk=1 και chunk=1000 είναι μικρές και δεν παρατηρείται κατάρρευση της απόδοσης για μικρά chunks. Αυτό δείχνει ότι ο μηχανισμός των μεγάλων αρχικών chunks πράγματι περιορίζει το scheduling overhead ενώ τα μικρότερα chunks στο τέλος βοηθούν στην πιο ισορροπημένη ανάθεση των βαριών επαναλήψεων στους διαθέσιμους πυρήνες. Στο συγκεκριμένο ανισοκαταναμημένο workload το guided λειτουργεί ως ένας πολύ καλός συμβιβασμός και προσφέρει καλύτερο load balancing από το static με μικρότερο κίνδυνο κακής επιλογής chunk σε σχέση με το dynamic.

Συνολική σύγκριση static, dynamic, guided

Για να γίνει πιο ξεκάθαρη η σύγκριση στον παραπάνω πίνακα παρουσιάζονται οι μέσοι χρόνοι για 4 threads και για διαφορετικά chunk sizes (1, 10, 100, 1000) για το scheduling στο ανισοκατανεμημένο different $f(x)$.

Schedule	chunk=1	chunk=10	chunk=100	chunk=1000
static	2.22	2.23	2.28	2.24
dynamic	3.69	2.34	2.25	2.25
guided	2.59	2.58	2.60	2.59



Η συνολική εικόνα δείχνει ότι σε αντίθεση με το ομοιόμορφο workload $f(x)=x^2$ στο ανισοκαταναμημένο different $f(x)$ οι διαφορές ανάμεσα στις πολιτικές είναι πιο ήπιες. Η `schedule(static)` εξακολουθεί να έχει το χαμηλότερο runtime overhead και δίνει χρόνους γύρω στα 2.2-2.3s σχετικά ανεξάρτητους από το chunk size. Ωστόσο δεν μπορεί να διορθώσει την ανισοκατανομή των βαριών επαναλήψεων οπότε η βελτίωση από την αύξηση των νημάτων είναι περιορισμένη σε σχέση με ότι θα περιμέναμε από ένα τέλεια ισοκαταναμημένο workload.

Η `schedule(dynamic)` με πολύ μικρό chunk 1 είναι σαφώς χειρότερη (3.7s για 4 threads) λόγω υπερβολικού scheduling overhead αφού κάθε επανάληψη αντιστοιχεί σε ένα μικροσκοπικό chunk και ο scheduler καλείται συνεχώς. Όταν όμως το chunk αυξηθεί σε 10-100-1000 ο χρόνος μειώνεται στην περιοχή 2.25-2.34s πολύ κοντά στο static καθώς το dynamic καταφέρνει να μοιράσει καλύτερα τις βαριές επαναλήψεις χωρίς το overhead να γίνεται κυρίαρχο.

Το `schedule(guided)` βρίσκεται ενδιάμεσα και για 4 threads δίνει χρόνους περίπου 2.58-2.60s για όλα τα chunk sizes ελαφρώς χειρότερους από static και dynamic αλλά εξαιρετικά σταθερούς ως προς το chunk. Αυτό δείχνει ότι τα μεγάλα αρχικά chunks περιορίζουν το overhead ενώ τα μικρότερα chunks προς το τέλος κρατούν το load balancing σε καλό επίπεδο χωρίς η απόδοση να εξαρτάται τόσο έντονα από τη συγκεκριμένη επιλογή του chunk size.

Συνολικά, για το ανισοκαταναμημένο different $f(x)$ μπορούμε να πούμε ότι:

- Το **static** παραμένει απλό με πολύ χαμηλό overhead και χρόνους ελαφρώς καλύτερους από guided και αντίστοιχους με ένα καλά ρυθμισμένο dynamic αλλά δεν είναι ιδανικό όταν το workload είναι πολύ άνισο.
- Το **dynamic** μπορεί να βελτιώσει ή να ταιριάζει την απόδοση του static εφόσον επιλεγεί κατάλληλο chunk size (π.χ. 10-100) αλλά είναι αρκετά ευαίσθητο στην επιλογή αυτή και τιμωρεί σκληρά πολύ μικρά chunks.
- Το **guided** προσφέρει έναν καλό συμβιβασμό και παρέχει αρκετά καλό load balancing και κρατά το scheduling overhead υπό έλεγχο και είναι πολύ πιο ανθεκτικό στις αλλαγές του chunk size με σχεδόν σταθερούς χρόνους σε όλο το εύρος των chunks.

Οδηγίες Εκτέλεσης

- **Compile:** `g++ -std=c++17 -fopenmp integration_openmp_schedules_different_fx.cpp -o integration_openmp_schedules_different_fx`
- **Run:** `./integration_openmp_schedules_different_fx <static|dynamic|guided> <chunk> <num_threads>`

Μέθοδος παραλληλοποίησης(Γ: Παραλληλοποίηση με OpenMP tasks)

Ανάλυση απόδοσης υλοποίησης με OpenMP Tasks χωρίς Αναδρομή

- Για την αξιολόγηση της υλοποίησης με OpenMP tasks εκτελέστηκαν πειράματα για 4 διαφορετικούς αριθμούς νημάτων (1, 2, 3 και 4 threads) και για 5 διαφορετικούς αριθμούς tasks (4, 8, 16, 32 και 64). Για κάθε συνδυασμό αριθμού tasks και threads πραγματοποιήθηκαν 5 επαναλήψεις και υπολογίστηκε ο μέσος χρόνος εκτέλεσης.
- με $f(x)$:

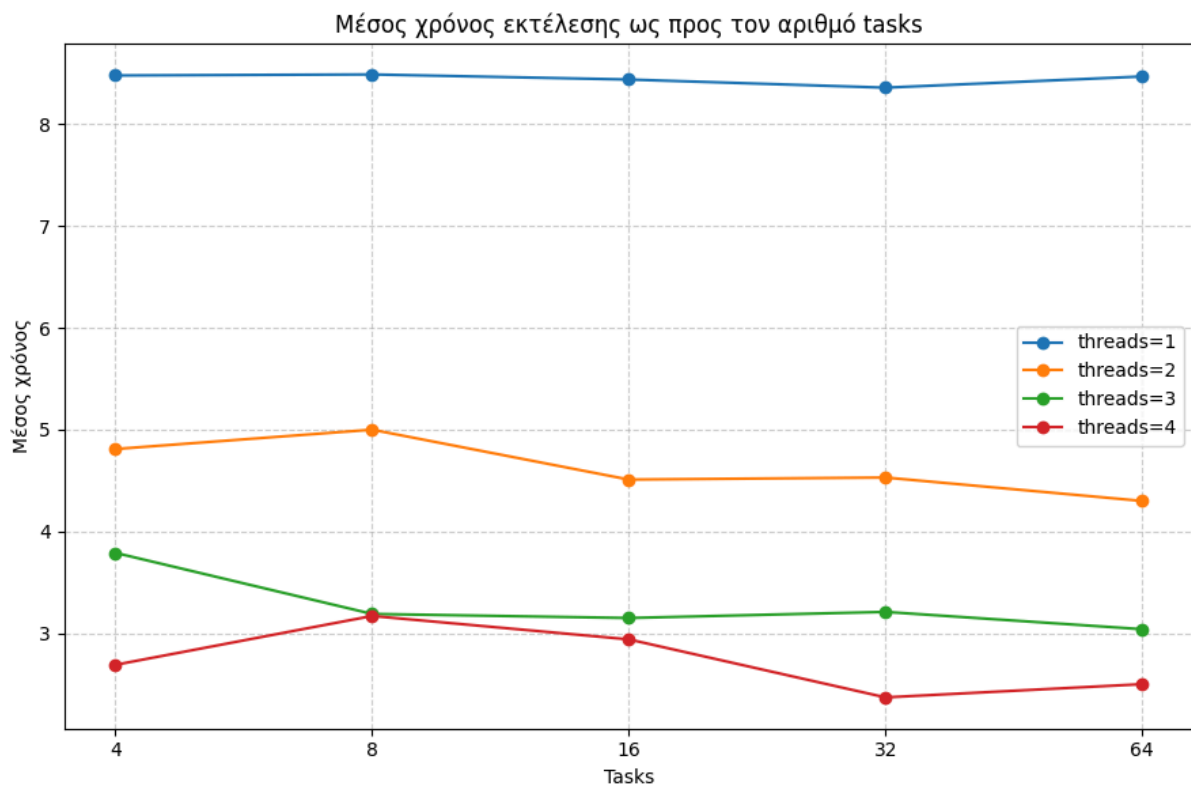
$$f(x) = \begin{cases} \sin^2(x) \sqrt{x+1}, & \text{αν } \lfloor x \rfloor \text{ είναι άρτιος} \\ x^2, & \text{αν } \lfloor x \rfloor \text{ είναι περιττός} \end{cases}$$

- Μέσοι χρόνοι εκτέλεσης χωρίς Αναδρομή

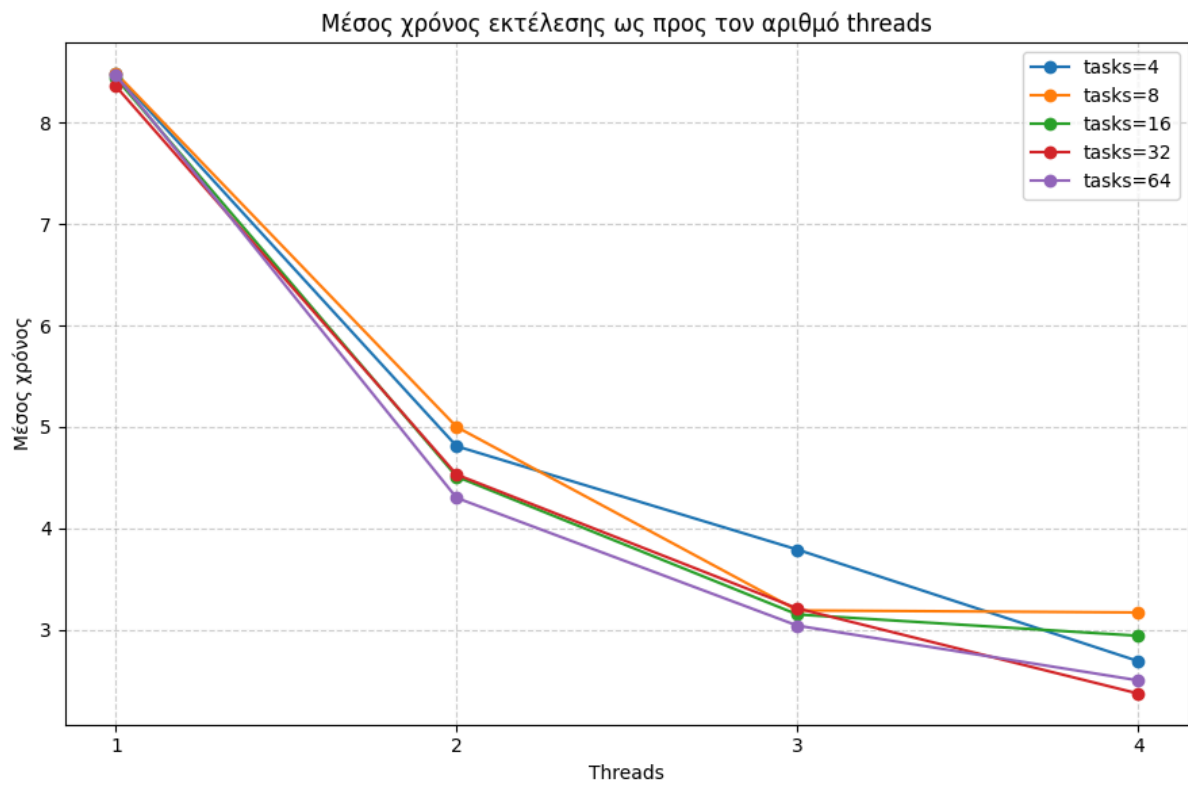
Tasks	Threads	Μέσος χρόνος
4	1	8.48
8	1	8.49
16	1	8.44
32	1	8.36
64	1	8.47
4	2	4.81
8	2	5.00
16	2	4.51
32	2	4.53
64	2	4.30
4	3	3.79
8	3	3.19

16	3	3.15
32	3	3.21
64	3	3.04
4	4	2.69
8	4	3.17
16	4	2.94
32	4	2.37
64	4	2.50

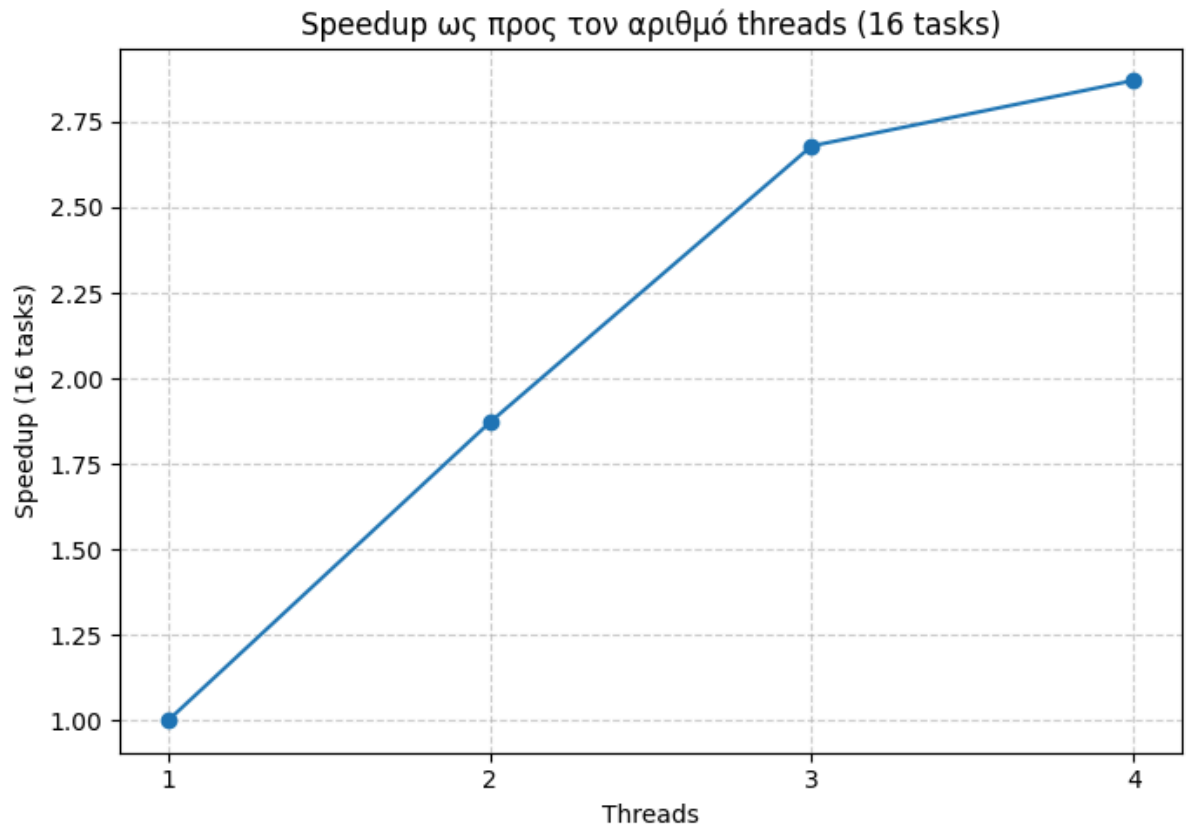
- **Μέσος χρόνος εκτέλεσης ως προς τον αριθμό tasks**



- **Μέσος χρόνος εκτέλεσης ως προς τον αριθμό threads**



- Speedup ως προς τον αριθμό threads



- Περίπτωση 1 thread
 - Στην περίπτωση εκτέλεσης με 1 thread η αύξηση του αριθμού των tasks δεν προσφέρει ουσιαστικό κέρδος. Οι χρόνοι παραμένουν γύρω στα 8.4-8.5s για όλες τις τιμές των tasks και οι διαφορές οφείλονται κυρίως σε θόρυβο μέτρησης. Αντίθετα όσο αυξάνεται ο αριθμός tasks αυξάνεται ελαφρά και το κόστος διαχείρισης από το runtime του OpenMP άρα με 1 thread εκτέλεση το tasking εισάγει κυρίως overhead χωρίς να προσφέρει βελτίωση load balancing.
- Περίπτωση 2 threads
 - Για 2 threads παρατηρείται σημαντική μείωση του χρόνου εκτέλεσης σε σχέση με το 1 thread όπου οι χρόνοι πέφτουν από ~8.4s σε περίπου 4.8-5.0s ανάλογα με τον αριθμό των tasks. Η καλύτερη επίδοση εμφανίζεται στα 64 tasks με μέσο χρόνο περίπου 4.30s ενώ για 16 και 32 tasks οι χρόνοι είναι γύρω στα 4.5s. Αυτό δείχνει ότι όσο περισσότερα tasks από threads βοηθά στη καλύτερη κατανομή του φόρτου μεταξύ των threads χωρίς το overhead δημιουργίας και διαχείρισης tasks να γίνεται απαγορευτικό.
- Περίπτωση 3 threads
 - Για 3 threads οι καλύτερες επιδόσεις εμφανίζονται περίπου στην περιοχή 8-16 tasks με χρόνους 3.19-3.15s ενώ για 32 tasks ο χρόνος

αυξάνεται ελαφρά στα 3.21s και για 64 tasks πέφτει ξανά στα περίπου 3.04s. Το αποτέλεσμα αυτό δείχνει ότι υπάρχει ένα σημείο ισορροπίας: πολύ λίγα tasks δεν εκμεταλλεύονται πλήρως τη δυναμική κατανομή εργασίας ενώ υπερβολικά πολλά tasks αυξάνουν το overhead και μπορούν να οδηγήσουν σε μεγαλύτερη μεταβλητότητα χωρίς μεγάλη βελτίωση.

- Περίπτωση 4 threads

- Στην περίπτωση των 4 threads που αντιστοιχεί και στον αριθμό των φυσικών πυρήνων του συστήματος οι χρόνοι κυμαίνονται περίπου από 2.3 μέχρι 3. s ανάλογα με τον αριθμό των tasks. Η καλύτερη επίδοση εμφανίζεται στα 32 tasks με μέσο χρόνο περίπου 2.37s ενώ πολύ κοντά βρίσκονται και οι ρυθμίσεις με 4, 16 και 64 tasks (2.69, 2.94 και 2.50s αντίστοιχα). Αυτό δείχνει ότι για το συγκεκριμένο πρόβλημα η επίδοση δεν εξαρτάται δραματικά από τον αριθμό tasks όταν το workload έχει ήδη κατανεμηθεί ικανοποιητικά στους διαθέσιμους πυρήνες και ότι ένα μέτριο πλήθος tasks ανά thread είναι αρκετό για καλό load balancing.

- Ανάλυση ως προς τον αριθμό threads (16 tasks)

- Για να μελετηθεί το scaling της υλοποίησης μπορεί να εξεταστεί ενδεικτικά η περίπτωση των 16 tasks. Ο μέσος χρόνος εκτέλεσης μειώνεται από 8.44s στο 1 thread σε 4.51s στα 2 threads 3.15s στα 3 threads και 2.94s στα 4 threads. Η προσθήκη του δεύτερου thread προσφέρει σημαντική βελτίωση (speedup ~1.87x) ενώ η μετάβαση στα 3 και 4 threads συνεχίζει να μειώνει τον χρόνο αλλά με ολοένα μικρότερα περιθώρια βελτίωσης καθώς εμφανίζεται ο περιορισμός από το σειριακό μέρος του κώδικα και το overhead του tasking.
- Η συμπεριφορά αυτή δείχνει ότι η υλοποίηση επιτυγχάνει speedup λίγο μικρότερο από 3x στη μετάβαση από 1 σε 4 threads για 16 tasks αλλά απέχει από το ιδανικό γραμμικό scaling. Αυτό είναι αναμενόμενο, καθώς στο συνολικό χρόνο εκτέλεσης συμμετέχουν όχι μόνο οι υπολογισμοί του ολοκληρώματος αλλά και το κόστος δημιουργίας των tasks ο συγχρονισμός με taskwait η χρήση atomic για τη συσσώρευση του integral και οι επιδράσεις της κοινόχρηστης μνήμης.

- Συμπεράσματα

- Η υλοποίηση με OpenMP tasks χωρίς αναδρομή έδειξε ότι η χρήση task-based παραλληλισμού μπορεί να βελτιώσει σημαντικά την απόδοση σε σχέση με τη μονονηματική εκτέλεση ιδιαίτερα όταν χρησιμοποιούνται 2 ή περισσότερα threads. Παράλληλα φάνηκε ότι η επιλογή του αριθμού tasks επηρεάζει την απόδοση μέσω της ισορροπίας ανάμεσα στο load balancing και στο overhead του runtime:

λίγα tasks περιορίζουν τη δυνατότητα δυναμικής κατανομής εργασίας, ενώ πάρα πολλά tasks αυξάνουν το κόστος διαχείρισης χωρίς ανάλογο κέρδος.

- Για το συγκεκριμένο πρόβλημα οι πιο αποδοτικές ρυθμίσεις εμφανίζονται γενικά στην περιοχή 16 έως 64 tasks ανάλογα με τον αριθμό των threads με την μεγαλύτερη σχετική βελτίωση να προκύπτει από τη μετάβαση από 1 σε 2 threads. Από εκεί και πέρα, η αύξηση των νημάτων και των tasks προσφέρει μεν καλύτερους χρόνους, αλλά με φθίνουσες αποδόσεις λόγω του αναπόφευκτου tasking overhead και των ορίων του διαθέσιμου παραλληλισμού.

Οδηγίες Εκτέλεσης

- **Compile**: `g++ -std=c++17 -fopenmp integration_tasks_no_recursion.cpp -o integration_tasks_no_recursion`
- **Run**: `.integration_tasks_no_recursion num_tasks threads`

Ανάλυση απόδοσης υλοποίησης με OpenMP Tasks με Αναδρομή

- Για την αναδρομική εκδοχή του κώδικα με OpenMP tasks εκτελέστηκαν πειράματα μεταβάλλοντας τον αριθμό των νημάτων από 1 έως 4. Η διάσπαση του ολοκληρώματος σε εργασίες γίνεται αναδρομικά μέσω της συνάρτησης `integrate_task`, η οποία δέχεται ένα εύρος επαναλήψεων `[i_start, i_end]` και είτε το επεξεργάζεται σειριακά είτε το χωρίζει στη μέση δημιουργώντας δύο νέα tasks (divide & conquer). Όταν το μήκος του εύρους γίνει μικρότερο από

ένα κατώφλι (threshold) η συνάρτηση υπολογίζει το ολοκλήρωμα σειριακά και προσθέτει το αποτέλεσμα στο κοινό άθροισμα με atomic. Για κάθε αριθμό threads πραγματοποιήθηκαν πολλαπλές επαναλήψεις και υπολογίστηκε ο μέσος χρόνος εκτέλεσης.

- με $f(x)$:

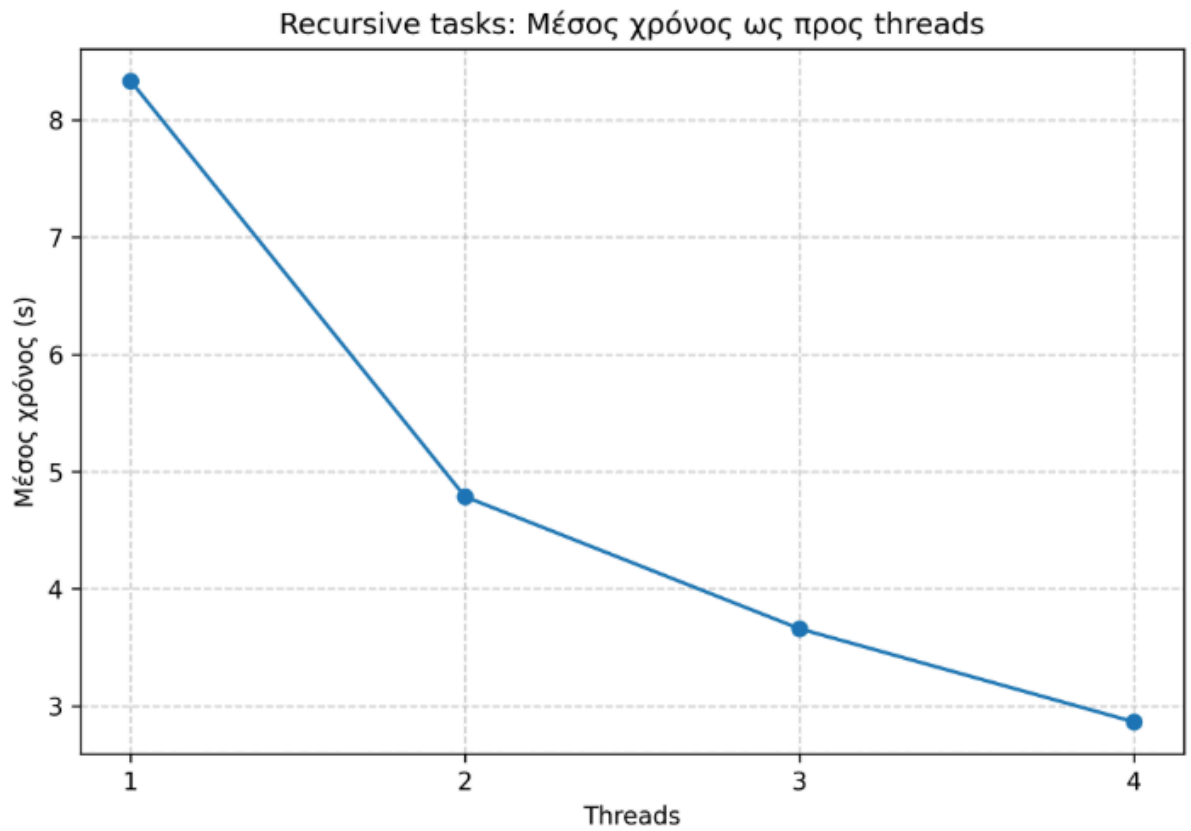
$$f(x) = \begin{cases} \sin^2(x) \sqrt{x+1}, & \text{αν } \lfloor x \rfloor \text{ είναι άρτιος} \\ x^2, & \text{αν } \lfloor x \rfloor \text{ είναι περιττός} \end{cases}$$

- Μέσοι χρόνοι εκτέλεσης (αναδρομική υλοποίηση)

Threads	Μέσος χρόνος
1	8.33325
2	4.78575
3	3.65965
4	2.86473

Παρατηρείται ότι ο χρόνος εκτέλεσης μειώνεται σημαντικά με την αύξηση των threads από περίπου 8.33s με 1 thread σε περίπου 2.86s με 4 threads γεγονός που δείχνει ότι η αναδρομική διάσπαση εκμεταλλεύεται ικανοποιητικά τον διαθέσιμο παραλληλισμό για το συγκεκριμένο μέγεθος προβλήματος.

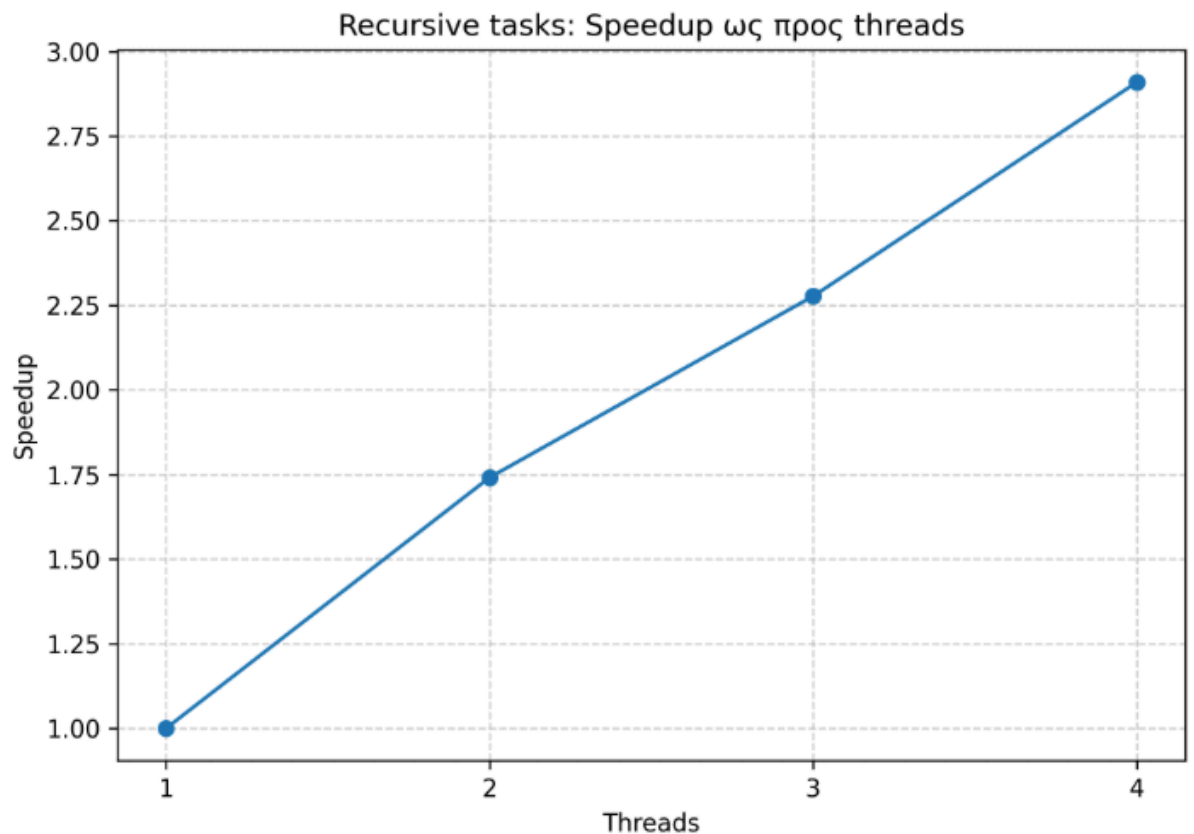
- Μέσος χρόνος (αναδρομικά tasks) ως προς τον αριθμό threads



- Speedup (αναδρομικά tasks) ως προς τον αριθμό threads.
Λαμβάνοντας ως βάση τον χρόνο με 1 thread προκύπτουν οι ακόλουθες τιμές speedup:

Threads	Speedup
1	1.00
2	1.74

3	2.28
4	2.91



Συμπεράσματα για την αναδρομική υλοποίηση με tasks

- Με την αναδιατύπωση της αναδρομικής υλοποίησης ώστε να καλύπτει το πλήρες εύρος επαναλήψεων $[0, N]$ και με κατάλληλη επιλογή του threshold η αναδρομική εκδοχή εκτελεί πλέον ουσιαστικά το ίδιο υπολογιστικό έργο με την non recursive υλοποίηση. Τα πειράματα δείχνουν ότι επιτυγχάνει σημαντική μείωση χρόνου εκτέλεσης καθώς αυξάνονται τα threads με speedup κοντά στο 3x για 4 threads αν και η απόδοση περιορίζεται από το αναπόφευκτο overhead του OpenMP runtime και το σειριακό τμήμα της αναδρομής.

Συνολικά η προσέγγιση divide and conquer με task nesting είναι συμβατή με τα ζητούμενα της εκφώνησης και προσφέρει καλή παραλληλοποίηση, υπό την προϋπόθεση ότι ο σχεδιασμός του threshold εξισορροπεί σωστά την ποσότητα εργασίας ανά task και το κόστος δημιουργίας και συγχρονισμού των tasks.

Οδηγίες Εκτέλεσης

- **Compile**: g++ -std=c++17 -fopenmp integration_tasks_recursion.cpp -o integration_tasks_recursion
- **Run**: .\integration_tasks_recursion num_tasks threads

Συγκριτική αξιολόγηση non recursive και recursive OpenMP tasks

Η non recursive υλοποίηση με OpenMP tasks, όπου ο αριθμός των tasks δίνεται ρητά ως παράμετρος, δείχνει καλή κλιμάκωση με τον αριθμό των νημάτων, ειδικά στη μετάβαση από 1 σε 2 και 3 threads, όταν επιλεγεί κατάλληλος αριθμός tasks (π.χ. 16-64) ώστε το granularity να εξισορροπεί το load balancing με το runtime overhead. Με αυτές τις ρυθμίσεις επιτυγχάνεται σημαντική μείωση χρόνου εκτέλεσης και speedup που προσεγγίζει τις ιδανικές τιμές για μικρούς αριθμούς threads, μέχρι το κόστος δημιουργίας και συγχρονισμού των tasks να γίνει συγκρίσιμο με το υπολογιστικό κόστος.

Η αναδρομική υλοποίηση με tasks, όταν διασπά το πλήρες εύρος επαναλήψεων $[0, N]$ με divide and conquer και κατάλληλο threshold εκτελεί ουσιαστικά το ίδιο υπολογιστικό έργο και εμφανίζει παρόμοια κλιμάκωση όπου ο μέσος χρόνος μειώνεται σχεδόν γραμμικά από 1 έως 4 threads με συνολικό speedup περίπου 2.9x ενώ η απόκλιση από το ιδανικό scaling οφείλεται κυρίως στο σειριακό τμήμα της

αναδρομής και στο overhead του OpenMP runtime (task creation, taskwait, atomic). Συνολικά τα αποτελέσματα αναδεικνύουν ότι οι non recursive tasks είναι ιδιαίτερα κατάλληλες όταν ο προγραμματιστής μπορεί να ελέγξει ρητά το granularity μέσω του num_tasks.